

Universidad Rafael Landívar

Campus Quetzaltenango

Licenciatura en Ingeniería en Informática y Sistemas

Estructura de Datos I — 2026

Catedrático: Ing. Juan Pablo Valiente

Manual Técnico

Generador de Imágenes por Capas — Proyecto Final

Estudiante:

Rodas Berreondo Ángel Emmanuel

Carné: 1594125

28 de mayo de 2026

1. Descripción General del Sistema

El Generador de Imágenes por Capas es una aplicación de consola escrita en Python que permite crear, gestionar y visualizar imágenes compuestas de múltiples capas de píxeles de color. El sistema organiza la información en estructuras de datos avanzadas implementadas manualmente (sin librerías externas de colecciones), incluyendo árboles binarios de búsqueda, listas circulares doblemente enlazadas y matrices dispersas.

La aplicación admite carga masiva desde archivos con formatos propios (.cap, .im, .usr), operaciones CRUD completas sobre capas, imágenes y usuarios, y generación de reportes gráficos mediante Graphviz.

Objetivos del sistema

- Almacenar píxeles (color + coordenada) de manera eficiente usando matrices dispersas.
- Gestionar un catálogo de capas con búsqueda $O(\log n)$ mediante un ABB.
- Organizar imágenes en una lista circular doblemente enlazada con orden por ID.
- Asociar imágenes a usuarios a través de un segundo ABB y listas enlazadas simples.
- Generar visualizaciones de las estructuras internas con Graphviz (.dot $\rightarrow$.png).

Requisitos del entorno

Componente	Versión mínima	Notas
Python	3.8+	Intérprete principal
Graphviz	cualquier estable	Comando dot en PATH del sistema
Sistema operativo	Windows / Linux / macOS	Compatible multiplataforma

2. Estructura del Proyecto

La organización de carpetas y archivos del proyecto es la siguiente:

```

proyecto/
├── main.py          # Punto de entrada, menú principal y lógica de UI
├── carga_masiva.py  # Parsers para archivos .cap, .im y .usr
├── estructuras/
│   ├── __init__.py
│   ├── matriz_dispersa.py # Implementación de Matriz Dispersa
│   ├── arbol_capas.py    # ABB de Capas + clase Capa
│   ├── lista_imagenes.py # Lista Circular Doble de Imágenes + Imagen
│   └── arbol_usuarios.py # ABB de Usuarios + clase Usuario
└── datos_ejemplo/
    ├── capas.cap        # Datos de capas de prueba
    ├── imagenes.im      # Datos de imágenes de prueba
    └── usuarios.usr     # Datos de usuarios de prueba

```

Responsabilidades por módulo

Archivo	Responsabilidad
main.py	Menú interactivo, flujo de la aplicación, estado global (arbol_capas, lista_imagenes, arbol_usuarios)
carga_masiva.py	Parseo de archivos .cap/.im/.usr con regex; validación de existencia de referencias cruzadas
estructuras/matriz_dispersa.py	Nodo de píxel con 4 punteros (arriba/abajo/izquierda/derecha); encabezados de fila y columna
estructuras/arbol_capas.py	NodoABB + ArbolCapas (insertar, buscar, eliminar, inorden/preorden/postorden); clase Capa con su MatrizDispersa
estructuras/lista_imagenes.py	NodoImagen + ListaCircularImagenes (insertar ordenado, buscar, eliminar); clase Imagen con ListaCapasImagen
estructuras/arbol_usuarios.py	NodoABBUsoario + ArbolUsuarios; clase Usuario con ListaImagenesUsuario (lista simple enlazada)

3. Estructuras de Datos Implementadas

3.1 Matriz Dispersa

Implementada en `matriz_dispersa.py`. Almacena únicamente los píxeles que tienen color asignado, evitando el desperdicio de memoria de una matriz densa. Cada celda con valor es un `NodoMatriz` conectado en cuatro direcciones.

Clases

Clase	Atributos clave	Descripción
NodoMatriz	fila, columna, color, arriba, abajo, izquierda, derecha	Nodo de píxel con 4 punteros de navegación
NodoEncabezado	indice, siguiente, anterior, acceso	Encabezado de fila o columna; acceso apunta al primer nodo
MatrizDispersa	cabeza_filas, cabeza_columnas	Raíz de las listas de encabezados

Operaciones principales

Método	Complejidad	Descripción
<code>insertar(fila, col, color)</code>	$O(f + c)$	Crea o actualiza el nodo en posición (fila,col)
<code>obtener(fila, col)</code>	$O(f + c)$	Retorna el color en (fila,col) o None
<code>superponer(otra)</code>	$O(p)$	Fusiona otra matriz encima (píxeles de otra tienen prioridad)
<code>obtener_todos_pixeles()</code>	$O(p)$	Lista de tuplas (fila, col, color)
<code>a_dict()</code>	$O(p)$	Diccionario {(f,c): color} para renderizado rápido

f = número de filas, c = número de columnas, p = número de píxeles existentes

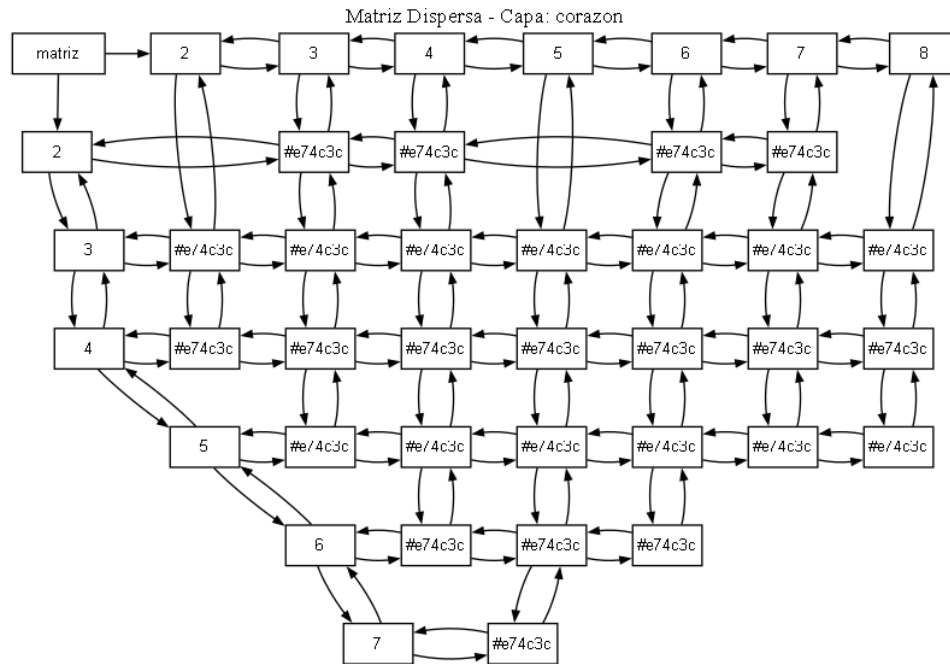


Fig. 3.1 — Visualización de la Matriz Dispersa de la capa 'corazon' generada por Graphviz

3.2 Árbol Binario de Búsqueda de Capas (ABB)

Implementado en `arbol_capas.py`. Almacena objetos Capa ordenados por su `id_capa` (string). Permite búsquedas, inserciones y eliminaciones en $O(\log n)$ promedio. Cada nodo contiene una referencia a la Capa, que a su vez contiene su MatrizDispersa.

Clases

Clase	Atributos	Descripción
Capa	<code>id_capa</code> , <code>matriz</code> (MatrizDispersa)	Unidad de capa con identificador único y sus píxeles
NodoABB	<code>capa</code> , <code>izquierdo</code> , <code>derecho</code>	Nodo del árbol binario de búsqueda
ArbolCapas	<code>raiz</code>	ABB principal de capas

Operaciones

Método	Complejidad	Descripción
<code>insertar(capa)</code>	$O(\log n)$	Inserta o actualiza una capa por <code>id_capa</code>
<code>buscar(id_capa)</code>	$O(\log n)$	Retorna el objeto Capa o

		None
eliminar(id_capa)	$O(\log n)$	Eliminación con sucesor inorden
inorden()	$O(n)$	Lista de capas ordenadas alfabéticamente
preorden() / postorden()	$O(n)$	Recorridos alternativos

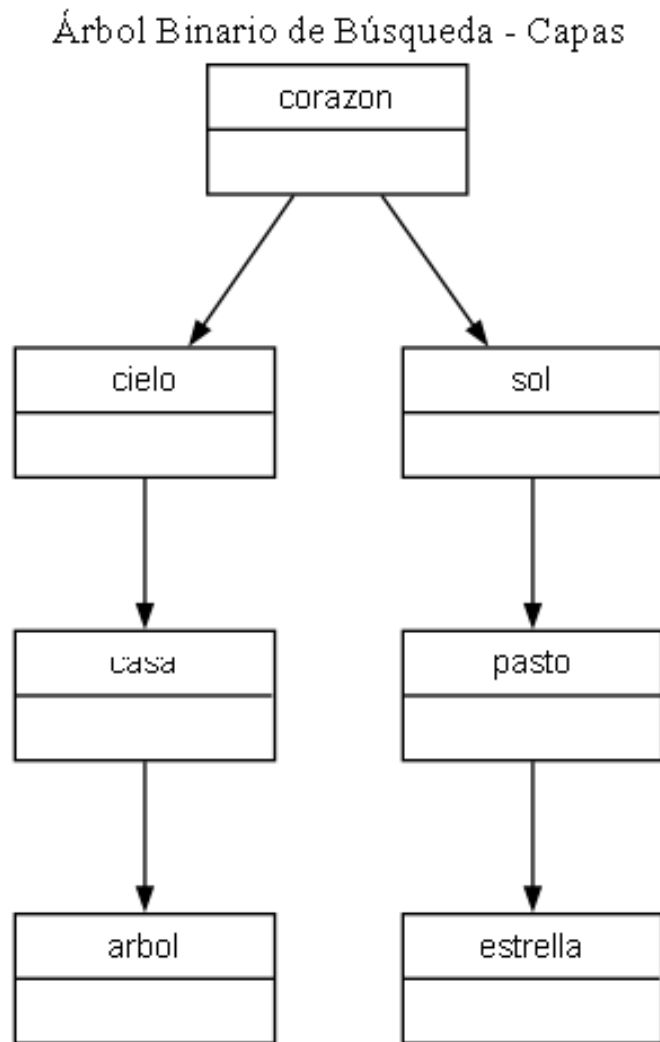


Fig. 3.2 — ABB de Capas con los datos de ejemplo

3.3 Lista Circular Doblemente Enlazada de Imágenes

Implementada en `lista_imagenes.py`. Almacena objetos `Imagen` ordenados por `id_imagen`. La estructura circular permite navegar hacia adelante y hacia atrás de forma continua. Cada

Imagen contiene una ListaCapasImagen (lista simple enlazada) cuyos nodos guardan referencias directas (punteros) a los objetos Capa en el ABB — no copias.

Clases

Clase	Atributos	Descripción
Imagen	id_imagen, lista_capas (ListaCapasImagen)	Imagen compuesta por capas en orden de superposición
NodoListaCapa	capa_ref, siguiente	Referencia a una Capa del ABB
ListaCapasImagen	cabeza, tamaño	Lista enlazada simple; orden = orden de superposición
NodoImagen	imagen, siguiente, anterior	Nodo de la lista circular
ListaCircularImagenes	cabeza, tamaño	Lista circular doblemente enlazada de imágenes

Operaciones

Método	Complejidad	Descripción
insertar_ordenado(imagen)	$O(n)$	Inserta manteniendo orden por id_imagen
buscar(id_imagen)	$O(n)$	Recorre la lista circular
eliminar(id_imagen)	$O(n)$	Elimina y reconstruye los enlaces circulares
obtener_todas()	$O(n)$	Lista de objetos Imagen en orden

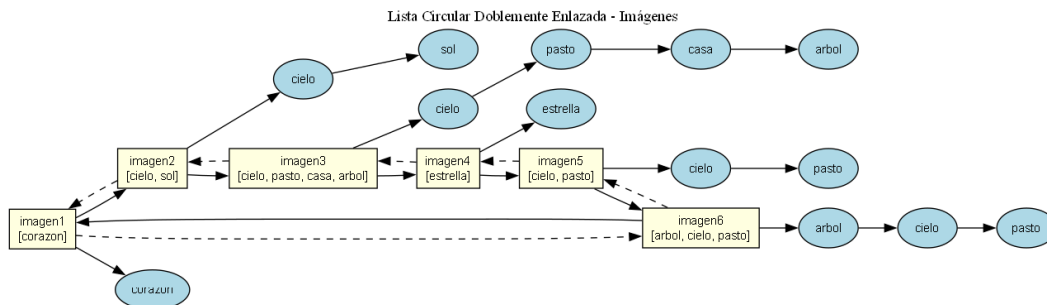


Fig. 3.3 — Lista Circular Doblemente Enlazada de Imágenes

3.4 Árbol Binario de Búsqueda de Usuarios

Implementado en arbol_usuarios.py. Almacena objetos Usuario ordenados por nombre (string). Cada Usuario tiene una ListaImagenesUsuario (lista simple enlazada) que guarda los IDs de las imágenes a las que tiene acceso.

Clases

Clase	Atributos	Descripción
Usuario	nombre, lista_imagenes (ListaImagenesUsuario)	Entidad de usuario con sus imágenes asociadas
NodoListaImagenUsuario	id_imagen, siguiente	Nodo de la lista de imágenes del usuario
ListaImagenesUsuario	cabeza	Lista simple enlazada de IDs de imagen
NodoABBUusuario	usuario, izquierdo, derecho	Nodo del ABB de usuarios
ArbolUsuarios	raiz	ABB principal de usuarios

Árbol Binario de Búsqueda - Usuarios

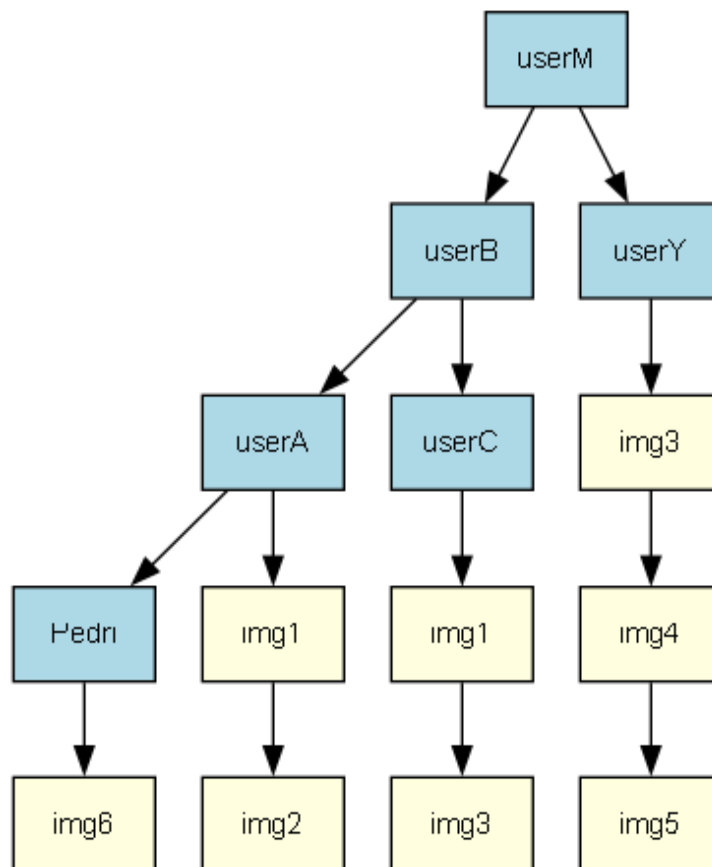
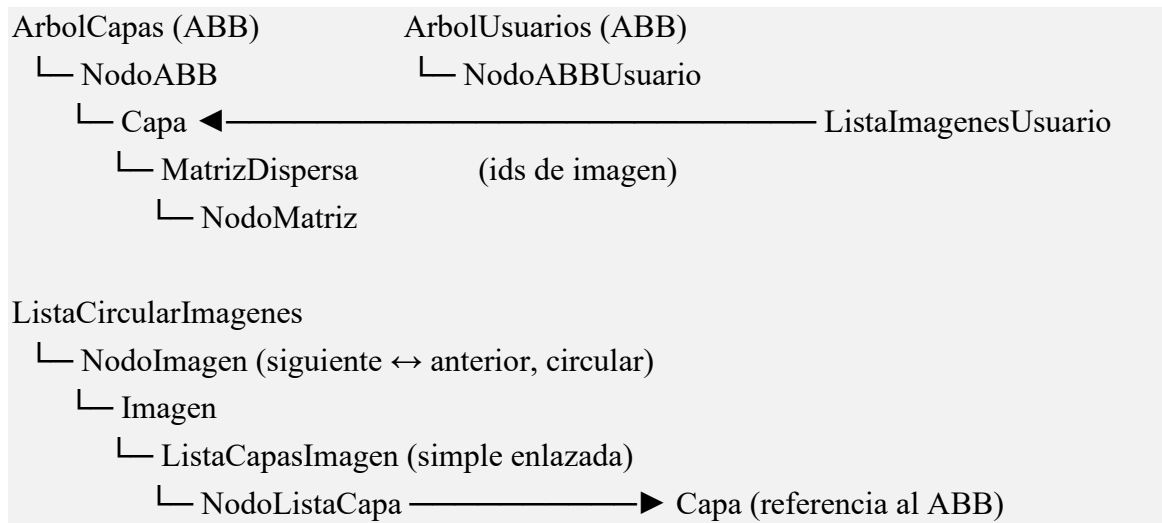


Fig. 3.4 — ABB de Usuarios con los datos de ejemplo

4. Diagrama de Relaciones entre Estructuras

El siguiente diagrama muestra cómo las estructuras de datos se relacionan entre sí. Las flechas indican referencias (punteros) entre objetos. Los objetos Capa en el ABB son compartidos: las imágenes no copian las capas, sino que apuntan a ellas.

Arquitectura de Datos



Punto clave: cuando una capa se modifica en el ABB, la modificación es visible automáticamente en todas las imágenes que la referencian, ya que los `NodoListaCapa` guardan referencias directas al mismo objeto `Capa` en memoria.

5. Módulo de Carga Masiva

El módulo `carga_masiva.py` contiene tres funciones de parseo que leen archivos con formatos propios del sistema. Todos los parsers eliminan comentarios de línea (`// ...`) antes de procesar.

5.1 Formato .cap — Capas

Cada bloque define una capa con sus píxeles:

```
corazon {
  2, 3, #e74c3c;
  3, 4, #e74c3c;
```

```
}
```

El parser usa expresiones regulares para extraer bloques id { ... } y luego parsea cada línea de píxel con separadores flexibles (coma o espacio entre fila y columna).

5.2 Formato .im — Imágenes

```
1 {  
    corazon;  
}  
3 {  
    cielo;  
    pasto;  
    casa;  
}
```

Cada imagen referencia capas por id. El parser verifica que cada id_capa exista en el ArbolCapas antes de agregar la referencia.

5.3 Formato .usr — Usuarios

```
userA:1,2;  
userY:3,4,5;
```

Formato nombre:id1,id2,...; El parser verifica que cada id de imagen exista en la ListaCircularImagenes antes de asociarla al usuario.

5.4 Orden de carga obligatorio

Paso	Archivo	Razón
1°	.cap	Las capas deben existir antes que las imágenes
2°	.im	Las imágenes deben existir antes que los usuarios
3°	.usr	Los usuarios referencian imágenes ya cargadas

6. Flujo de la Aplicación (main.py)

main.py expone un menú jerárquico de consola. El estado global se mantiene en tres variables de módulo: arbol_capas, lista_imagenes y arbol_usuarios.

Menú principal

Opción	Descripción
1. Carga Masiva	Submenú para cargar .cap, .im, .usr individualmente o en conjunto
2. Capas	CRUD: insertar, buscar, modificar, eliminar, listar capas
3. Imágenes	CRUD: insertar, buscar, modificar (agregar/quitar capas), eliminar, listar
4. Usuarios	CRUD: insertar, buscar, modificar, eliminar, listar
5. Reportes	Generación de gráficas con Graphviz (árboles, matrices, imágenes)
6. Salir	Termina la aplicación

Generación de imágenes compuestas

Para renderizar una imagen, el sistema recorre la ListaCapasImagen de la Imagen en orden, y llama a MatrizDispersa.superponer() de manera acumulativa. Las capas posteriores en la lista sobrescriben los píxeles de las anteriores en posiciones coincidentes.

```
resultado = MatrizDispersa()
for capa_ref in imagen.lista_capas.obtener_capas_en_orden():
    resultado.superponer(capa_ref.matriz)
# resultado contiene la imagen final renderizada
```

7. Reportes y Visualización

El sistema genera archivos .dot (Graphviz) y los convierte a .png mediante el comando dot. Los reportes se guardan en la carpeta reportes/ y se abren automáticamente con el visor predeterminado del sistema.

Tipos de reporte disponibles

Reporte	Descripción
Árbol de Capas	Visualización del ABB con todos los nodos y sus id_capa
Árbol de Usuarios	Visualización del ABB de usuarios y sus

	listas de imágenes
Lista de Imágenes	La lista circular completa con los ids de imagen
Matriz dispersa de capa	Grafo de nodos de una capa con sus píxeles coloreados
Imagen renderizada	Resultado final de superponer todas las capas de una imagen
Recorridos del ABB	Inorden, preorden y postorden del árbol de capas

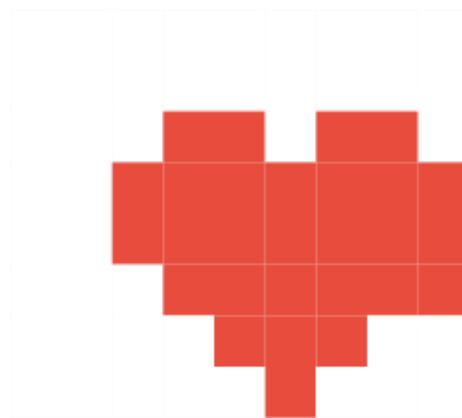


Fig. 7.1 — Imagen 1 renderizada (capa 'corazon')

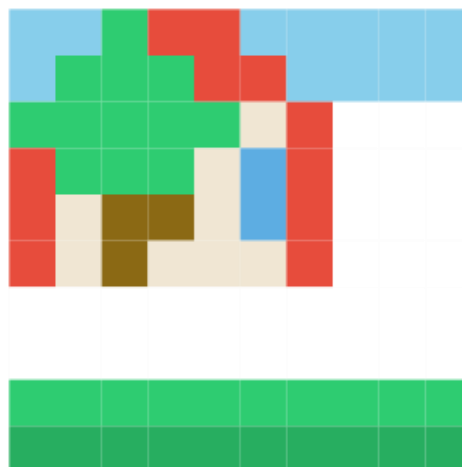


Fig. 7.2 — Imagen 3 renderizada (cielo + pasto + casa + arbol)

8. Análisis de Complejidad

Operación	Estructura	Tiempo	Espacio
Insertar/buscar/eliminar capa	ABB Capas	$O(\log n)$ prom.	$O(n)$
Insertar/buscar/eliminar usuario	ABB Usuarios	$O(\log n)$ prom.	$O(n)$
Insertar/buscar imagen	Lista Circular	$O(n)$	$O(n)$
Insertar píxel	Matriz Dispersa	$O(f + c)$	$O(p)$
Superponer matrices	Matriz Dispersa	$O(p)$	$O(1)$
Renderizar imagen completa	$k \text{ capas} \times p \text{ píxeles}$	$O(k \cdot p)$	$O(p_{\text{total}})$
Recorrido inorden ABB	ABB Capas/Usuarios	$O(n)$	$O(h)$

$n = \text{nodos en árbol}$, $f = \text{filas}$, $c = \text{columnas}$, $p = \text{píxeles existentes}$, $k = \text{capas en imagen}$, $h = \text{altura del árbol}$

9. Instrucciones de Ejecución

Instalación de dependencias

1. Instalar Python 3.8 o superior.
2. Instalar Graphviz y agregar el ejecutable dot al PATH del sistema:

```
# Windows: descargar desde https://graphviz.org/download/  
# Linux:  
sudo apt-get install graphviz  
# macOS:  
brew install graphviz
```

Ejecución del sistema

```
cd proyecto  
python main.py
```

Carga de datos de ejemplo

Dentro del menú, seleccionar Carga Masiva → Carga Completa e ingresar las rutas:

```
datos_ejemplo/capas.cap  
datos_ejemplo/imagenes.im
```

10. Glosario

Término	Definición
ABB	Árbol Binario de Búsqueda. Árbol donde cada nodo tiene a su izquierda valores menores y a su derecha valores mayores.
Matriz Dispersa	Estructura que solo almacena los elementos no nulos de una matriz, ahorrando memoria.
Lista Circular	Lista enlazada donde el último nodo apunta al primero, formando un ciclo.
Capa (Layer)	Conjunto de píxeles con coordenada (fila, columna) y color hexadecimal asociados a un ID único.
Imagen	Composición de una o más capas superpuestas en orden específico.
Superponer	Combinar dos matrices dispersas: los píxeles de la segunda sobrescriben los de la primera.
Graphviz	Herramienta de código abierto para generar diagramas de grafos a partir de archivos .dot.
Inorden	Recorrido de árbol: subárbol izquierdo → raíz → subárbol derecho. En un ABB produce orden ascendente.